

PRAKTEK TEKNOLOGI IOT II

LAPORAN PRAKTEK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum

Mata Kuliah Teknologi IOT II



DISUSUN OLEH :

NAMA : Raffi Maulana Abdillah

NPM 25782021

PRODI : TEKNOLOGI REKAYASA INTERNET

KELAS : 3A

DOSEN : Dr. ir. Septafiansyah Dwi Putra, S. T.,M.T.

: Agus Ambarwari, S.Pd.,M.Kom.

MENTRI PENDIDIKAN DAN KEBUDAYAAN PROVINSI LAMPUNG

POLITEKNIK NEGERI LAMPUNG

TAHUN AJARAN 2025 / 2026

LATIHAN:

Latihan 1 (Pemahaman Jaringan)

Soal:

Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan Station (STA) pada kedua praktikum di atas, & bukan mode Access Point (AP)?

Jawaban:

Mode Station dipilih karena kedua NodeMCU perlu berada dalam 1 jaringan Wi-Fi yang sama agar bisa saling berkomunikasi. Jika menggunakan Access Point (AP), tiap NodeMCU akan buat jaringan sendiri, sehingga perangkat tidak bisa saling terhubung kecuali ada mekanisme *bridging* tambahan. Dengan STA, NodeMCU bisa terhubung ke router yang sama sehingga komunikasi Client-Server lancar.

Latihan 2 (Modifikasi Antarmuka & Telemetry)

Soal:

Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan & masih memakai 2 tombol terpisah (ON + OFF). Lakukan 2 modifikasi berikut pada Web Server:

1. Menampilkan Kelembapan:

Tambah pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Sisipkan placeholder `%HUMIDITY%` pada kerangka HTML di dalam tag `<div class="sensor-box">`, lalu perbarui fungsi `handleRoot()` menggunakan `html.replace("%HUMIDITY%", ...)` agar nilai kelembapan muncul di halaman web.

o Potongan HTML Tambahan (Cetak Kelembapan):

HTML

```
<div class="sensor-box">
  <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celsius</p>
  <p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
</div>
```

2. Tombol Dinamis Tunggal (Toggle Button):

Ubah antarmuka agar hanya menggunakan 1 buah tombol tunggal yang statusnya adaptif mengikuti kondisi Relay saat ini:

- o Jika Relay **OFF**, tombol menampilkan tulisan "**NYALAKAN (ON)**" (misal: warna hijau/biru) & mengarahkan aksi ke `/relay/on`.
- o Jika Relay **ON**, tombol otomatis berubah menjadi tulisan "**MATIKAN (OFF)**" (misal: warna merah) & mengarahkan aksi ke `/relay/off`.
- o **Fungsi `handleRoot()`:**

C++

```
void handleRoot() {
  String html = index_html;
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (isnan(t)) html.replace("%TEMPERATURE%", "--");
  else html.replace("%TEMPERATURE%", String(t));

  if (isnan(h)) html.replace("%HUMIDITY%", "--");
  else html.replace("%HUMIDITY%", String(h));

  if (digitalRead(relayPin) == LOW) {
    html.replace("%RELAY_BUTTON%", "<a href=\"/relay/on\"><button class=\"btn-on\">NYALAKAN (ON)</button></a>");
```

```

    } else {
        html.replace("%RELAY_BUTTON%", "<a href=\"/relay/off\"><button class=\"btn-off\">MATIKAN (OFF)</button></a>");
    }

    server.send(200, "text/html", html);
}

```

Latihan 3 (Analisis Kode Client)

Soal:

Pada Praktikum 2, Serial Monitor Client mencetak HTTP Response Code. Analisis-lah apa yang akan dicetak oleh Serial Monitor Anda jika ternyata NodeMCU Server teman Anda mendadak kehilangan daya (*offline*)?

Jawaban:

Jika Server NodeMCU kehilangan daya, maka Client tetap mencoba melakukan `http.GET()`. Hasilnya, Serial Monitor akan mencetak nilai negatif yang menandakan koneksi gagal / Server tidak ditemukan.

Latihan 4 (Pengembangan Algoritma)

Soal:

Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga Client mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35 °C (bukan menggunakan LDR). Tulis baris algoritmanya!

Jawaban:

```

C++
float suhu = dht.readTemperature();

if ((WiFi.status() == WL_CONNECTED) && (suhu > 35)) {
    WiFiClient client;
    HTTPClient http;

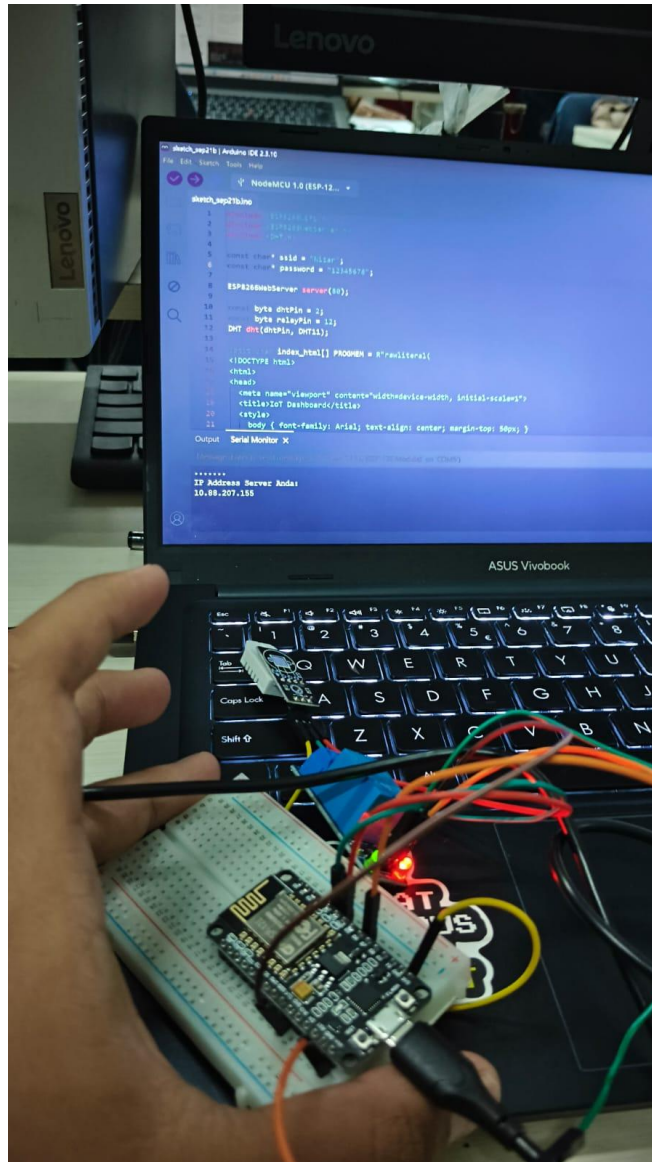
    http.begin(client, serverName);
    int httpStatusCode = http.GET();

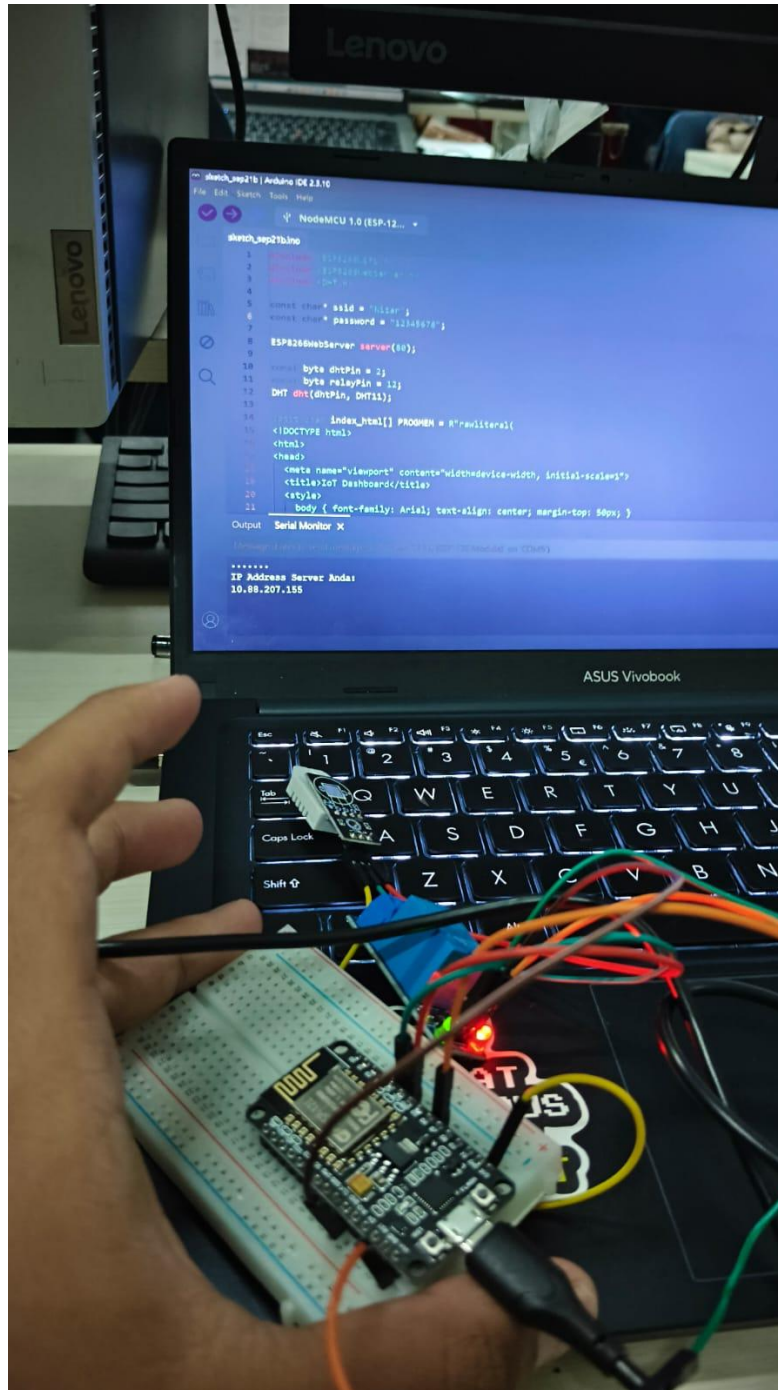
    Serial.print("HTTP Response Code: ");
    Serial.println(httpStatusCode);

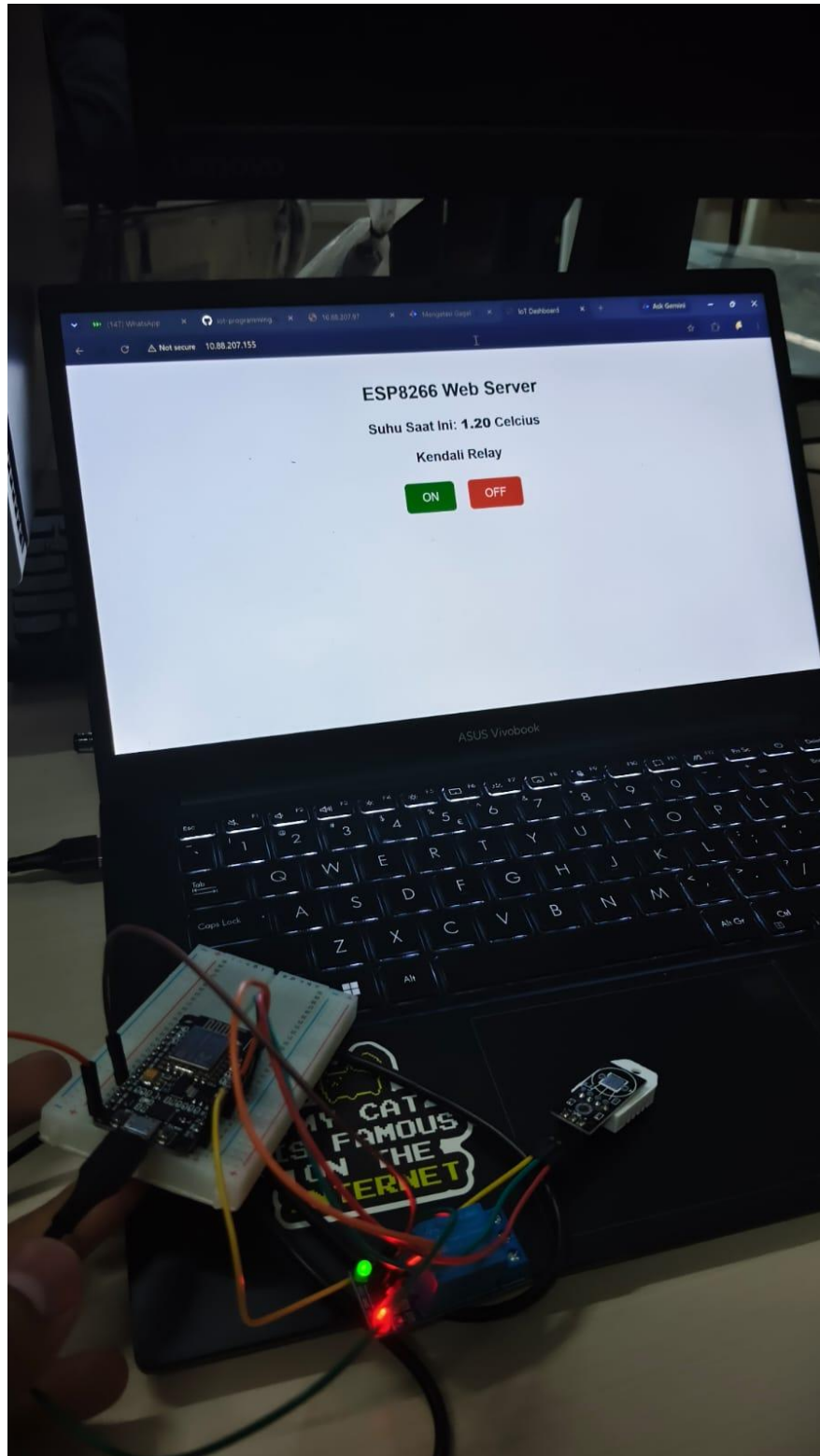
    http.end();
    delay(10000);
}

```

PRAKTIKKUM 1:







PENJELASAN PRAKTIKKUM 1:

1. Langkah Kerja Rangkaian (Hardware)

- Sensor DHT: Hubungkan pin DATA/OUT/S ke D4 (GPIO 2), VCC ke 3V3, dan GND ke GND.
- Modul Relay: Hubungkan pin IN/Signal ke D6 (GPIO 12), VCC ke VIN/VU NodeMCU, dan GND ke GND.

2. Langkah Kerja Pemrograman (Software)

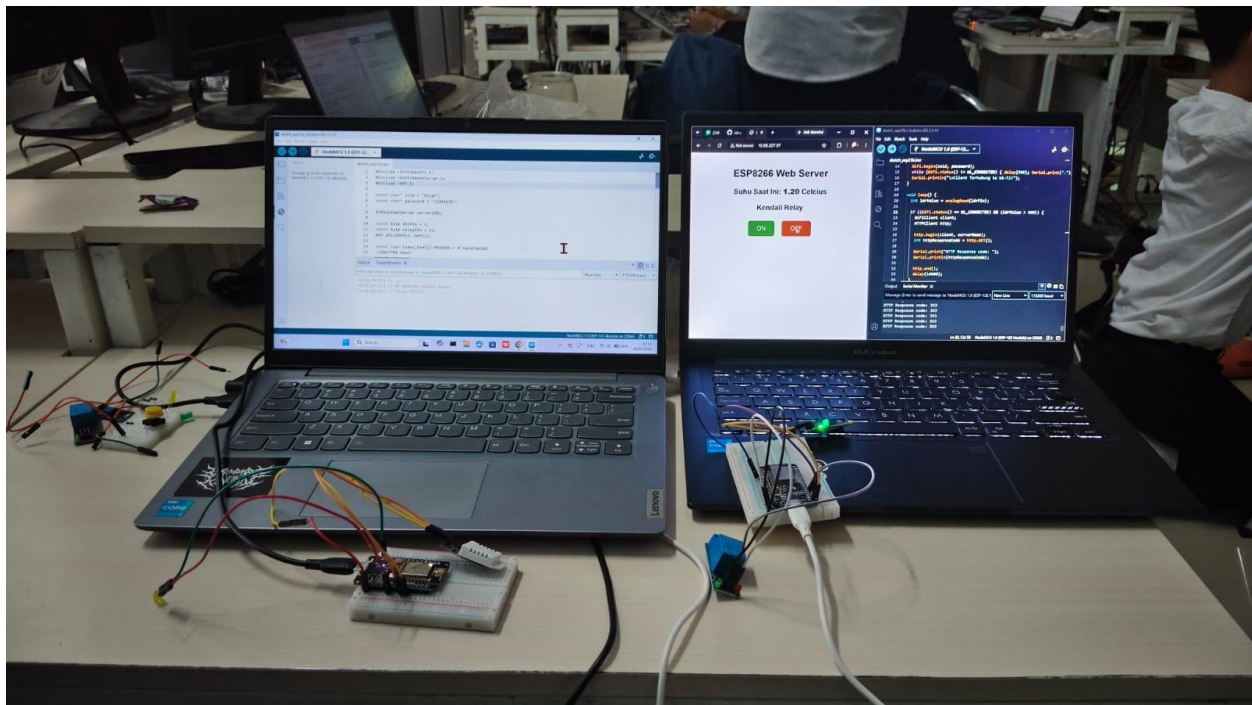
1. Buat Sketch Baru: Buat berkas baru di Arduino IDE, lalu masukkan kode program.
2. Konfigurasi Wi-Fi: Sesuaikan variabel ssid dan password Wi-Fi yang digunakan.
3. Teknik Kode: Kode antarmuka HTML multibaris ditulis menggunakan teknik *Raw String Literal C++* (R"rawliteral(...)rawliteral") agar tidak perlu menambahkan tanda kutip di setiap baris HTML.
4. Upload: Unggah (*upload*) program ke NodeMCU.

3. Penyelaian & Pengujian (Hasil Akhir)

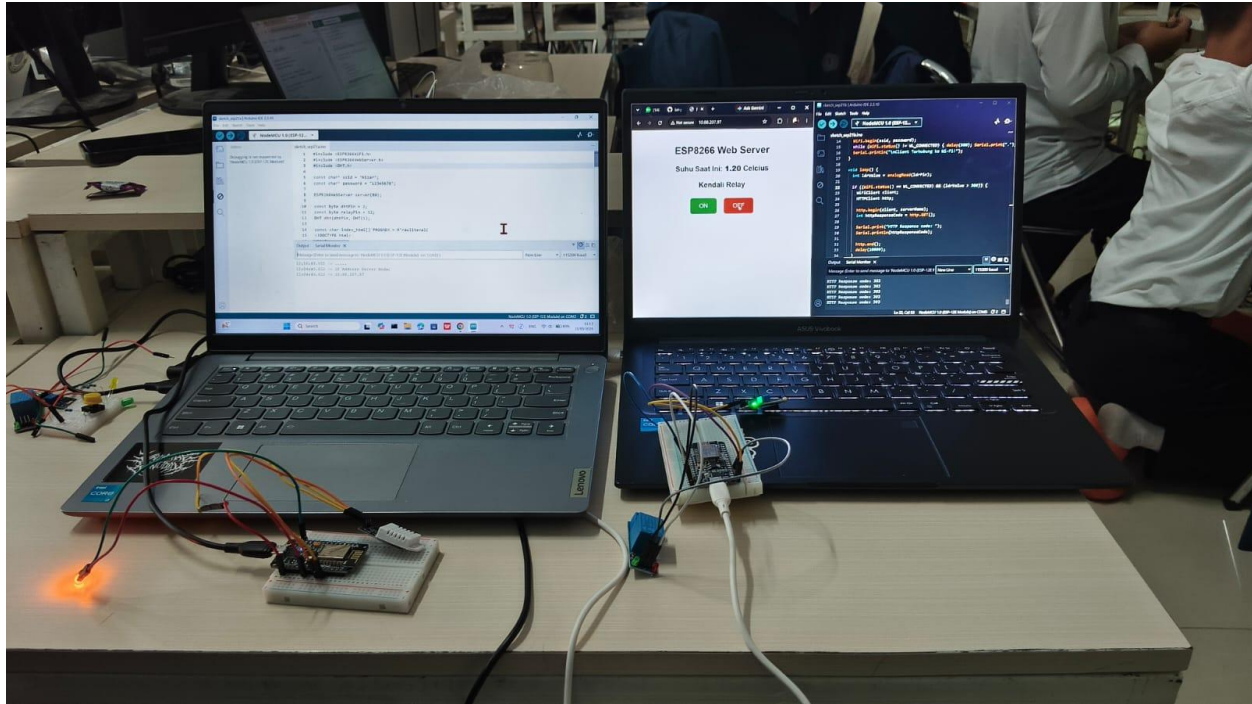
1. Cek Alamat IP: Buka Serial Monitor pada Arduino IDE, lalu catat alamat IP NodeMCU yang didapatkan (misal: 192.168.43.15).
2. Koneksi Perangkat: Hubungkan laptop/HP ke jaringan Wi-Fi yang sama.
3. Akses Web Server: Buka *browser* (Chrome/Safari) dan ketikkan alamat IP NodeMCU.
4. Pengujian Fitur:
 - Melihat Data: Halaman web akan menampilkan suhu dari sensor DHT secara dinamis (menggunakan mekanisme `html.replace()` untuk mengganti placeholder `%TEMPERATURE%`).
 - Kontrol Relay: Klik tombol ON dan OFF di antarmuka web. Jika berhasil, relay akan merespons (berbunyi *cetek*) melalui perintah *routing* (`server.on("/relay/on", ...)` dan `/relay/off`).

PRAKTIKKUM 2 :

SAAT INTENSITAS CAHAYA TINGGI LED AKAN MATI:



KETIKA CAHAYA REDUP LED AKAN MENYALA:



*NOTED: Relay juga bisa di on atau di off kan melalui server\

PENJELASAN PRAKTIKKUM 2:

Skenario

NodeMCU kedua difungsikan sebagai Client yang terhubung ke sensor LDR. Ketika LDR mendeteksi kondisi gelap, NodeMCU Client akan mengirimkan sinyal/permintaan HTTP secara jarak jauh melalui jaringan Wi-Fi untuk menyalakan relay yang ada pada NodeMCU Server (dari Praktikum 1).

1. Langkah Kerja Rangkaian (Hardware)

- Rangkai sensor LDR dan Resistor 10k Ohm menggunakan skema pembagi tegangan (*voltage divider*).
- Hubungkan jalur sinyal keluaran pembagi tegangan tersebut ke pin analog A0 pada NodeMCU.

2. Langkah Kerja Pemrograman (Software)

1. Buat Sketch Baru: Buka proyek baru di Arduino IDE.
2. Atur Target Server & Wi-Fi: Masukkan kode program, lalu ubah variabel `serverName` dengan IP Address NodeMCU Server (dari Praktikum 1). Pastikan kedua NodeMCU terhubung ke jaringan Wi-Fi yang sama persis.
3. Upload: Unggah program ke NodeMCU Client dan buka Serial Monitor.

3. Penyelesaian & Cara Kerja Kode (Hasil Akhir)

- Pengujian: Tutup sensor LDR dengan tangan hingga gelap. NodeMCU Client akan mendeteksi perubahan kondisi ini dan otomatis mengirimkan perintah *request* ke Server, sehingga relay pada NodeMCU Server akan menyala.
- Logika Kode yang Bekerja:
 - `WiFiClient client; & HTTPClient http;`: Membentuk objek komunikasi HTTP (berperan layaknya *browser* pada laptop).
 - `http.begin(client, serverName);`: Menentukan alamat URL sasaran milik

NodeMCU Server.

- o `int httpResponseCode = http.GET();` Mengirimkan permintaan HTTP GET ke Server. Nilai respons akan bernilai positif (seperti 200 atau 303) jika berhasil, atau bernilai negatif jika terjadi *error/Server offline*.
- o `delay(10000);` Jeda waktu selama 10 detik (*cooldown*) agar Client tidak membombardir (*flood*) Server dengan permintaan *ON* secara terus-menerus saat kondisi masih gelap.

TUGAS MANDIRI:

- Antarmuka (User Interface) IoT tidak boleh statis. Saat ini, Web Server kita tidak mampu memperbarui informasi suhunya secara otomatis ke layar pengunjung tanpa diperintah. Kita harus menyelesaikannya.
- Skenario Proyek (Web Dashboard): Anda akan bertindak sebagai Full-Stack IoT Developer untuk memutakhirkan kapabilitas Web Server pada Praktikum 1, sembari mengevaluasi limitasi arsitektur tradisional HTTP.
- Tujuan: Mampu memanipulasi struktur kerangka HTML mentah yang tersimpan di memori NodeMCU untuk mencapai interaktivitas tampilan otomatis (Auto-Refresh).
- Tingkat Kesulitan: Menengah o Instruksi Perangkaan:
- Lakukan modifikasi kode hanya pada sisi NodeMCU Server (Praktikum 1).
- Modifikasi variabel Raw Literal `index.html`. Sisipkan perintah Meta Refresh HTML (sebuah Tag standar web kuno namun ampuh) persis di dalam cakupan tag untuk memaksa browser memuat ulang halaman secara sekuensial setiap 5 detik.
- int (Petunjuk): Sintaks standar yang harus disisipkan adalah . Setelah selesai, buka browser, dan diamati layarnya. Biarkan selama 20 detik tanpa menyentuh mouse/keyboard.
- Instruksi Analisis Arsitektur: i). Setelah Auto-Refresh berjalan, cobalah tekan tombol Relay ON / OFF dari browser.
- Apa kelemahan visual/interaksi (flickering) yang Anda alami saat menggunakan metode Meta Refresh HTTP konvensional seperti ini untuk sistem yang mengklaim diri sebagai Real Time Dashboard? Tulis evaluasi logis Anda di Laporan Praktikum (Anda bisa membahas masalah bandwidth, layar berkedip putih, atau potensi bentrokan jika Anda mengklik tepat di detik ke-5).

Jawab: kelemahan visualnya yaitu selain layar akan berkedip tiap saat, sistem yang otomatis update juga akan menggunakan lebih banyak bandwidth dan data internet sehingga akan mengganggu pengalaman tampilan interaktif pada UI pengguna.

Kriteria Keberhasilan: - Nilai suhu di browser otomatis berganti seiring suhu ruangan tanpa perlu mengklik panah refresh browser secara manual. - Jawaban analisis mengenai kelemahan protokol HTTP GET statis tertuang jelas di dokumentasi tugas.

KODE TUGAS MANDIRI:

```
sketch_sep21a | Arduino IDE 2.3.10
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12E M...

sketch_sep21a.ino
1 #include <ESP8266WiFi.h>
2 #include <ESP8266WebServer.h>
3 #include <DHT.h>
4
5 const char* ssid = "Nizar";
6 const char* password = "12345678";
7
8 ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12 DHT dht(dhtPin, DHT11);
13
14 const char index_html[] PROGMEM = R"rawliteral(
15 <!DOCTYPE html>
16 <html>
17 <head>
18   <meta name="viewport" content="width=device-width, initial-scale=1">
19   <meta http-equiv="refresh" content="5">
20   <title>IoT Dashboard</title>
21   <style>
22     body { font-family: Arial; text-align: center; margin-top: 50px; }
23     button { padding: 15px 30px; font-size: 20px; border-radius: 8px; margin: 10px; cursor: pointer; }
24     .btn-on { background-color: #4CAF50; color: white; border: none; }
25     .btn-off { background-color: #f44336; color: white; border: none; }
26     .sensor-box { font-size: 24px; font-weight: bold; }
27   </style>
28 </head>
29
30 <body>
31   <h1>ESP8266 Web Server</h1>
32   <div class="sensor-box">Suhu: %TEMPERATURE% &deg;C</div>
33   <p>
34     <a href="/relay/on"><button class="btn-on">ON</button></a>
35     <a href="/relay/off"><button class="btn-off">OFF</button></a>
36   </p>
37 </body>
38 </html>
39 )rawliteral";
40
41 void handleRoot() {
42   String html = index_html; // Salin kerangka HTML ke variabel dinamis
43   float t = dht.readTemperature();
44
45   // Mengganti teks placeholder %TEMPERATURE% dengan suhu nyata
46   if (isnan(t)) {
47     html.replace("%TEMPERATURE%", "--");
48   } else {
49     html.replace("%TEMPERATURE%", String(t));
50   }
51   server.send(200, "text/html", html);
52 }
53
54 void handleRelayOn() {
55   digitalWrite(relayPin, HIGH);
56   server.sendHeader("Location", "/");
57
58 void handleRelayOff() {
59   digitalWrite(relayPin, LOW);
60   server.sendHeader("Location", "/");
61   server.send(303);
62 }
63
64 void setup() {
65   Serial.begin(115200);
66   pinMode(relayPin, OUTPUT);
67   digitalWrite(relayPin, LOW);
68   dht.begin();
69
70   WiFi.mode(WIFI_STA);
71   WiFi.begin(ssid, password);
72   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
73   Serial.println("\nIP Address Server Anda: ");
74   Serial.println(WiFi.localIP());
75
76   server.on("/", handleRoot);
77   server.on("/relay/on", handleRelayOn);
78   server.on("/relay/off", handleRelayOff);
79   server.begin();
80 }
81
82 void loop() {
83   server.handleClient();
84 }
85
86 }
```

SCREENSHOOT DAN HASIL FOTO TUGAS MANDIRI:



